OFICINA DE GIT E GITHUB

COMMIT ZERO

Seu primeiro voo no controle de versão



Matheus S. Gomes

INSTITUTO FEDERAL
São Paulo
Campus Capivari

COMMIT ZERO

Seu primeiro voo no controle de versão

Autor

Matheus Santos Gomes

Edição

Edição Oficina 2026, versão 1.0, outubro de 2026

Uso

Material de apoio à oficina

Avisos e Créditos

Material independente. Esta apostila não tem vínculo com o GitHub. GitHub e Octocat são marcas registradas do GitHub, Inc. O Git é um software livre.

Arte do Octocat: Ilustrações oficiais do Octodex (octodex.github.com), criadas pelos artistas Cameron McEfee (Octonaut, Jetpacktocat, Setuptocat, Repo e Linktocat) e Gabe Cook (Welcometocat e Codercat). Marcas registradas do GitHub, Inc.

Referâncias de estudo: artigo "O que é Git e GitHub", da Alura; livro Pro Git (git-scm.com/book); documentação do GitHub (docs.github.com).



Octocat

Esta apostila foi feita para ser lida com o terminal aberto ao lado. Ler Git sem digitar é como estudar natação em terra firme.

Sumário

Missão 01

O Caos do Código e o Nascimento do Git ..... Pág. 5

Por que o Git existe, o modelo distribuído e a diferença entre Git e GitHub.

Missão 02

Equipando a Nave ..... Pág. 9

Instalação, verificação de sistemas (git --version) e configuração de identidade (git config).

Missão 03

Conceitos Fundamentais ..... Pág. 12

O repositório, o que é um commit e o mapa das três áreas (Hangar, Doca e Cofre).

Missão 04

Grimório de Comandos Locais ..... Pág. 15

O ciclo completo: git init, status, add, commit, log e diff.

Missão 05

A Estação Espacial GitHub ..... Pág. 20

Conectando à nuvem, git remote add, push, clone, pull e o README.

Como usar esta apostila



Octocat

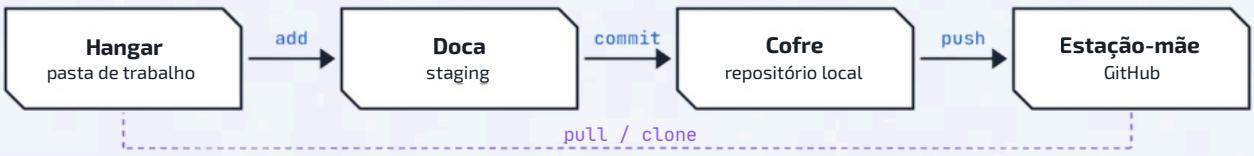
Bem-vindo à tripulação! Eu sou o Octocat e vou te acompanhar. Prometo comandos bons e piadas de qualidade duvidosa...

Como ler os comandos

- O símbolo **$** é só o prompt do terminal. Você **não** digita ele.
- Trechos entre **< >** são espaços para você trocar, por exemplo **git switch <nome-da-branch>**. Não digite os sinais de menor e maior.
- Maiúsculas e minúsculas importam. Termine cada comando com Enter
- O Git está em inglês por padrão. Se o seu estiver em português, as mensagens saem traduzidas, mas os comandos são os mesmos.

O Mapa da Nave

Você vai ver este mapa várias vezes. Ele mostra por onde o seu trabalho viaja. Cada comando é uma comporta entre dois lugares.



Missão 01

O Caos do Código e o Nascimento do Git

Por que o Git existe e por que você vai querer usá-lo pelo resto da carreira.



Objetivo	Entender o problema que o Git resolve
Comandos novos	Nenhum (a decolagem é de ideias)



Octocat

Antes de ligar os motores, uma pergunta: você já salvou um arquivo como trabalho_final_v2_agora_vai.docx? Se sim, você já conhece o problema que vamos resolver.

O drama dos arquivos

Imagine que você está construindo um site. Depois de algumas semanas, a sua pasta está assim:

meu-site

```
meu-site/
    index.html
    index_final.html
    index_final_v2.html
    index_agora_vai.html
    index_agora_vai_DEFINITIVO.html
```

Responda sem abrir nenhum arquivo: qual é a versão mais recente? O que mudou de uma para a outra? E se você precisar voltar à primeira? Esse é o **caos do código**: sem um sistema para registrar as mudanças, a única memória do projeto são os nomes dos arquivos, e nomes de arquivo mentem.

Um **sistema de controle de versão** resolve isso: guarda o histórico de todas as alterações, registra quem fez cada uma e deixa você voltar a qualquer momento do projeto.

Tradutor Galáctico

Termo real	Metáfora	Onde a metáfora falha
Controle de versão	O diário de bordo da nave	Um diário só registra. O controle de versão também deixa você viajar de volta a qualquer página dele.

Quando o trabalho é em equipe

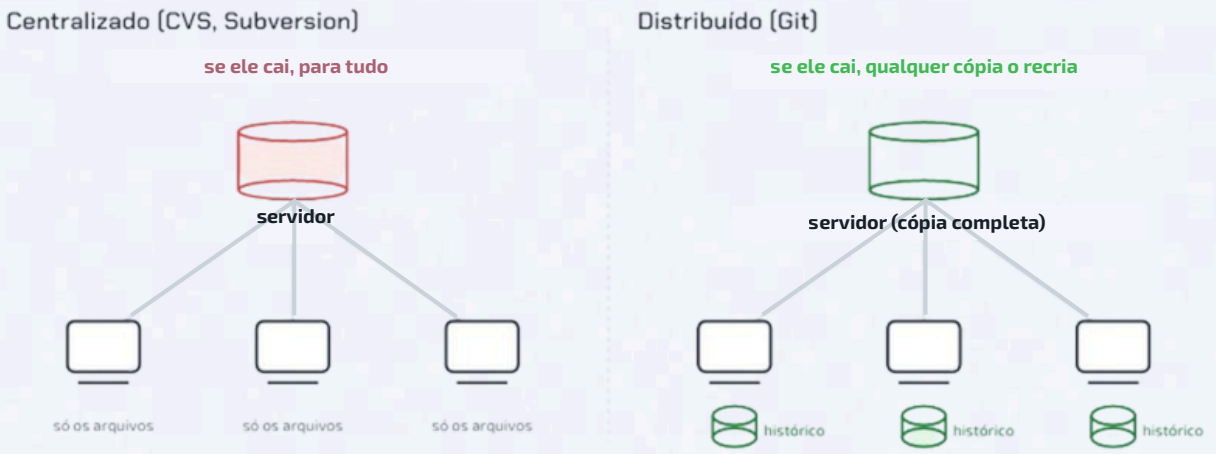
Antes dos computadores, os projetos viviam em pilhas de papel. Depois passaram para arquivos digitais, e várias pessoas começaram a mexer no mesmo arquivo em um servidor central. Parecia o paraíso, até duas pessoas editarem a mesma coisa ao mesmo tempo. Havia dois desfechos, ambos ruins:

- **Sobrescrita:** quem salva por último apaga o trabalho de quem salvou antes.
- **Bloqueio:** o arquivo trava para todo mundo enquanto uma pessoa edita, e a equipe fica esperando.

E havia um terceiro problema, mais silencioso: sem histórico, ninguém sabia quem tinha alterado o quê.

Duas gerações de controle de versão

Ferramentas como **CVS** e **Subversion** resolveram o problema do histórico: guardavam tudo e registravam o autor de cada mudança. O ponto fraco era a arquitetura **centralizada**: tudo morava em um único servidor. Se ele falhasse, ninguém versionava nada. E se o disco corrompesse sem backup, o histórico do projeto inteiro ia embora.



No modelo **distribuído**, existe um repositório no servidor, mas **cada pessoa guarda uma cópia completa do projeto, com todo o histórico**. O Git é assim. O que isso muda:

- Você trabalha sem internet, porque o seu repositório local tem tudo.
- Se o servidor quebrar, qualquer cópia pode restaurá-lo.
- Pode haver vários repositórios remotos, para fluxos de trabalho diferentes.
- Cada pessoa trabalha de forma independente e em paralelo.

Octocat

Imagine que a Estação-mãe explode. Sem pânico: cada nave da frota carrega o mapa estelar completo. Basta uma delas reconstruir a base.



O nascimento do Git

O **Linux** é um dos sistemas mais usados do mundo, e o seu núcleo (o kernel) é mantido por gente espalhada pelo planeta. Gerenciar tantas contribuições sem uma boa ferramenta é um pesadelo. Por isso, o projeto passou a usar uma ferramenta de controle de versão distribuído chamada **BitKeeper**.

anos 1990 — o kernel cresce com gente do mundo todo

2002 — o projeto adota o BitKeeper

2005 — o acordo acaba e nasce o Git

Em 2005, a relação entre a comunidade do kernel e a empresa dona do BitKeeper azedou e o uso gratuito acabou. O criador do Linux, Linus Torvalds, precisava de um substituto e, em vez de procurar um pronto, escreveu o seu: o Git. Os objetivos eram claros:

- Ser **rápido**;
- Funcionar bem com **projetos enormes**, como o próprio Linux;
- Suportar desenvolvimento **não linear**, com milhares de ramificações em paralelo;
- Ser **distribuído.**

Por que o Git conquistou o mundo?

Existem outras ferramentas, como o **Mercurial** e o **Helix Core**. Mas o Git ficou gigante porque é código aberto e gratuito, é veloz, suporta ramificações e projetos grandes e, talvez o principal, existe o **GitHub**, que tornou o compartilhamento muito simples.

Circuito Interno

Do que o Git é feito? Muitos comandos de alto nível nasceram como scripts de shell, o que amarrava a ferramenta a sistemas parecidos com o Linux. Para rodar bem em outras plataformas, como o Windows, boa parte foi reescrita em linguagem C, ganhando desempenho.

Git x GitHub

Essa confusão é tão comum que merece uma tabela.

	Git	GitHub
O que é?	Ferramenta de controle de versão, usada pela linha de comando	Plataforma na web para hospedar repositórios e colaborar
Onde vive	Na sua máquina	Em servidor na nuvem
Precisa de internet?	Não, para o trabalho local	Sim
Serve para	Registrar o histórico do projeto	Compartilhar, revisar código em equipe e divulgar seu trabalho
É obrigatório?	É a base de tudo	Não: existem GitLab, Bitbucket e outras plataformas



Octocat

O Git é a câmera e o diário de bordo. O GitHub é a estação espacial onde o diário fica guardado e onde outros tripulantes podem consultá-lo.

Teste Relâmpago

1. O que costuma acontecer quando várias pessoas editam o mesmo arquivo em um servidor central sem controle de versão?

a) Nada, tudo funciona bem
b) O trabalho de alguém pode ser sobrescrito, ou o arquivo ficar bloqueado
c) O servidor apaga o arquivo
d) O arquivo vira PDF

a) Não registrarem o autor
b) Serem incompatíveis com Windows
c) Depender de um único servidor central
d) Não permitirem voltar a versões antigas

3. No modelo distribuído, o que cada pessoa tem na própria máquina?

a) Só os arquivos de versão atual
b) Um atalho para o servidor
c) Nada, tudo fica na nuvem
d) Uma cópia completa do repositório, com o histórico

4. Qual afirmação está correta?

a) O Git é a ferramenta e o GitHub é uma plataforma que a utiliza
b) Git e GitHub são o mesmo programa
c) O GitHub foi criado antes do Git
d) O Git só funciona dentro do GitHub

5. Qual destes é um serviço alternativo ao GitHub?

a) Subversion
b) Linux
c) GitLab
d) Shell

Cápsula do capítulo

- Controle de versão guarda o histórico do projeto e quem fez cada mudança.
- Sistemas centralizados dependem de um único servidor. No modelo distribuído, cada pessoa tem uma cópia completa.
- O Git foi criado por Linus Torvalds em 2005: distribuído, rápido e gratuito.
- Git é a ferramenta. GitHub é a plataforma que a usa.

Missão 02

Equipando a Nave

Instalar o Git, perder o medo do terminal e se identificar como astronauta.



Objetivo	Ter o Git instalado e configurado, e uma conta no GitHub
Comandos novos	git --version, git config



Octocat

Nenhuma nave decola sem checklist. Vamos instalá-la, aprender os comandos de terminal que bastam para começar e dizer ao Git quem é você.

Instalando o Git

Windows

1. Acesse **git-scm.com** e baixe o instalador do Git para Windows.
2. Execute o instalador e avance com as opções padrão.
3. Abra o **Git Bash** pelo menu Iniciar. É o terminal que vamos usar na apostila.

Linux (Debian e Ubuntu)

Terminal

```
$ sudo apt update
$ sudo apt install git
```

Em outras distribuições: **sudo dnf install git** (Fedora) ou **sudo pacman -S git** (Arch).

macOS

Abra o Terminal e digite **git --version**. Se o Git não estiver instalado, o macOS oferece instalar as Xcode Command Line Tools: aceite. Outra opção é o Homebrew: **brew install git**.

Para conferir que deu certo, em qualquer sistema:

Git Bash

```
$ git --version
git version 2.47.0
```

O número da versão pode ser outro. O importante é aparecer uma mensagem assim e não um erro.

Terminal 101

O Git vive no terminal. Você não precisa dominar tudo, só oito comandos:

Comando	O que faz	Exemplo
`pwd`	Mostra em que pasta você está	`pwd`
`ls`	Lista o que há na pasta	`ls`
`cd <pasta>`		`cd Documentos`
`cd ..`	Volta uma pasta	`cd ..`
`mkdir <nome>`	Cria uma pasta	`mkdir diario-de-bordo`
`clear`	Limpa a tela	`clear`
Tab	Completa o nome que você começou a digitar	digite `cd Doc` e aperte Tab
Seta para cima	Traz de volta o comando anterior	aperte e depois Enter.

Octocat

A tecla Tab é a minha favorita. Ela digita por você e ainda evita erro de ortografia. Se o terminal responder com um erro, respire: quase sempre é o nome da pasta.



Dizendo ao Git quem você é

Todo commit leva o nome e o e-mail de quem o fez. Configure uma vez:

Git Bash

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email "voce@email.com"
$ git config --global init.defaultBranch main
$ git config --list
user.name=Seu Nome
user.email=voce@email.com
init.defaultbranch=main
```

- Use o **mesmo e-mail da sua conta do GitHub**, para os commits ficarem ligados ao seu perfil.
- **--global** vale para todos os seus projetos. Sem essa opção, dentro de um repositório, a configuração vale só para ele (**--local**).
- **init.defaultBranch main** faz os projetos novos nascerem com a branch **main**, o mesmo nome que o GitHub usa.

Em computador de laboratório, a configuração pode sumir quando a máquina reinicia. Se o Git reclamar que não sabe quem você é, repita estes comandos.

Criando a conta no GitHub

1. Acesse **github.com** e clique em **Sign up**.
2. Informe o e-mail, crie uma senha e escolha um nome de usuário.
3. Resolva a verificação e digite o código que chegará ao seu e-mail.
4. Escolha o plano gratuito. Ele basta para tudo nesta apostila.

Os nomes dos botões podem mudar com o tempo, mas o caminho é esse. O seu nome de usuário vira parte do endereço dos seus projetos, então escolha um que você não tenha vergonha de pôr no currículo.

Como o GitHub reconhece você?

O GitHub **não aceita mais a senha da conta** para enviar código com o Git. No primeiro **git push**, o Git para Windows abre uma janela do navegador para você entrar (Git Credential Manager). Outras opções são um token de acesso pessoal ou uma chave SSH, que veremos no capítulo 9.
Em computador compartilhado, **não crie tokens** e saia da conta ao terminar.

Teste Relâmpago

1. Qual comando mostra as configurações do Git?

a) **git show**
b) **git config --list**
c) **git status**
d) **git init**

2. Para que serve a opção --global?

a) Vale só para o projeto atual
b) Vale para todos os projetos do usuário
c) Apaga o histórico
d) Envia ao GitHub

3. O que a tecla Tab faz no terminal?

a) Fecha o terminal
b) Completa o nome que você começou a digitar
c) Limpa a tela
d) Volta uma pasta

Cápsula do capítulo

- ✔ Instale o Git e confira com **git --version**.
- ✔ Oito comandos de terminal bastam: **pwd, ls, cd, mkdir, clear** e mais as teclas Tab e seta para cima.
- ✔ Configure **user.name**, **user.email** e **init.defaultBranch**.
- ✔ Crie a conta no GitHub com o mesmo e-mail.

Missão 03

Conceitos Fundamentais

As poucas ideias que sustentam todo o resto: repositório, commit e as três áreas.

Objetivo	Entender o que é um commit e por onde o trabalho passa
Comandos novos	nenhum (vamos desenhar o mapa)





Octocat

Se você entender este capítulo, o resto do Git deixa de ser mágica. Faça com calma e, se puder, desenhe o mapa no papel.

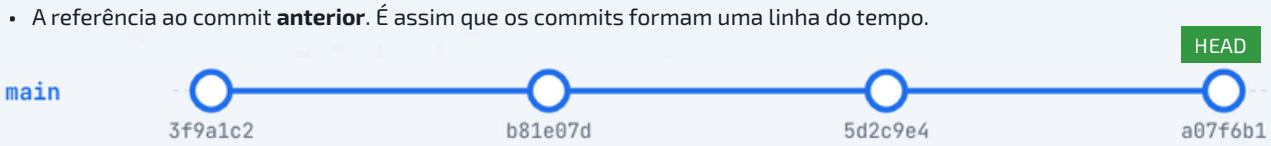
O repositório

Um **repositório** é uma pasta de projeto que o Git acompanha. Por dentro, ele guarda o histórico completo. Esse histórico mora em uma pasta escondida chamada **.git** (vamos explorá-la no capítulo 5).

O commit: uma foto com legenda

Um **commit** é um registro do estado do projeto em um momento, como uma fotografia colada no diário de bordo, com legenda. Cada commit guarda:

- O **retrato** dos arquivos naquele momento (um snapshot);
- **Quem** fez, **quando** e uma **mensagem** explicando o motivo;
- Um **identificador único**, o hash, algo como **3f9a1c2**;
- A referência ao commit **anterior**. É assim que os commits formam uma linha do tempo.



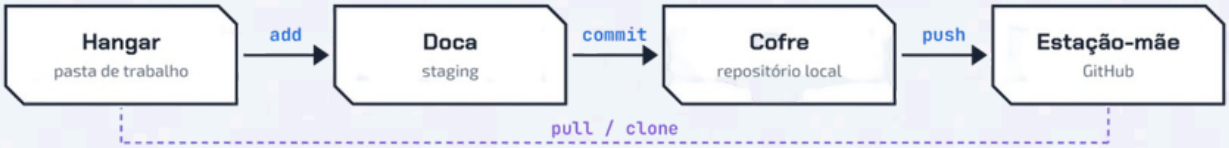
O HEAD é o marcador "você está aqui": aponta para o commit em que o projeto está agora.

Tradutor Galáctico

Termo real	Metáfora	Onde a metáfora falha
Commit	Foto colada no diário de bordo	Uma foto é só uma imagem. O commit é um estado completo do projeto, ao qual você pode voltar.
Hash	Número de série da foto	Não é sequencial: é calculado a partir do conteúdo.

As três áreas do Git

Aqui está a ideia mais importante do Git. O seu trabalho passa por **três lugares** antes de virar histórico:



Área	Nome Técnico	O que é
Hangar	Working directory (diretório de trabalho)	A pasta do projeto, onde você edita os arquivos de verdade.
Doca	Staging area (área de preparação)	Onde você separa o que vai entrar na próxima foto. Você escolhe, arquivo por arquivo.
Cofre	Repositório local	O histórico de commits guardado no seu computador, na pasta **.git**.
Estação-mãe	Repositório remoto	Uma cópia na nuvem, como a do GitHub. É opcional e só entra no capítulo 9.



Octocat

Por que a Doca existe? Para você montar commits organizados. Você pode ter mexido em cinco arquivos e registrar só dois, com uma mensagem que explica aqueles dois.

Circuito Interno

Arquivos têm estado. Para o Git, cada arquivo está não rastreado (ele ainda não conhece), modificado (mudou desde o último commit), preparado (está na Doca) ou commitado (está seguro no Cofre). O comando git status mostra isso o tempo todo.

Ramificações: o aperitivo

Uma **branch** é uma linha de commits paralela. Serve para testar uma ideia sem mexer na linha principal, a **main**. Quando a ideia dá certo, as linhas se encontram em um **merge**. Você vai pôr a mão nisso no capítulo 7. Por enquanto, guarde a imagem:



1. Onde você escolhe o que entra no próximo commit?

a) Working directory
b) Staging area
c) Repositório remoto
d) Pasta .git

2. O que é o HEAD?

a) O primeiro commit
b) O marcador do commit em que o projeto está agora
c) Uma branch remota
d) O nome do autor

3. Um commit guarda:

a) Só a mensagem
b) Apenas os arquivos alterados
c) O estado do projeto, autor, data, mensagem e referência ao commit anterior
d) Somente o hash

Cápsula do capítulo

- Um repositório é uma pasta que o Git acompanha, com o histórico em **.git**.
- Um commit guarda o estado do projeto, o autor, a data, a mensagem e um hash.
- O trabalho passa por Hangar (working directory), Doca (staging) e Cofre (repositório local).
- Branches são linhas paralelas que depois se encontram em um merge.

Missão 04

Grimório de Comandos Locais

Os primeiros feitiços: criar, preparar, registrar e consultar.



Objetivo	Fazer o ciclo completo do Git no seu computador
Comandos novos	init, status, add, commit, log, diff



Octocat

Se você decorar um único comando, que seja **git status**. Ele diz onde você está e qual é o próximo passo. Eu o uso o tempo todo.

O ciclo de voo

Todo o trabalho local com Git segue o mesmo ciclo. Ele se repete dezenas de vezes por dia:

1. **Criar** o projeto e iniciar o Git (**git init**), uma única vez.
2. **Editar** os arquivos normalmente, no seu editor.
3. **Conferir** o que mudou (**git status, git diff**).
4. **Preparar** o que vai na foto (**git add**).
5. **Registrar** a foto (**git commit**).
6. **Consultar** a linha do tempo (**git log**).



git init: cria o universo local

Git Bash

```
$ mkdir diario-de-bordo
$ cd diario-de-bordo
$ git init
Initialized empty Git repository in .../diario-de-bordo/.git/
```

Cria a pasta escondida **.git** e transforma a pasta atual em um repositório. Rode **uma vez** por projeto, e só dentro da pasta do projeto.

git status: o scanner do Octocat

Git Bash

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
```

Mostra em qual branch você está, o que mudou e em que área cada arquivo está. Não altera nada, então pode rodar à vontade.

git add: leva o arquivo do Hangar para a Doca

Git Bash

```
$ git add index.html
$ git add index.html style.css
$ git add .
```

O primeiro prepara um arquivo, o segundo prepara vários e o terceiro prepara **tudo** que mudou na pasta atual. Depois do **git add**, o **git status** passa a mostrar o arquivo em Changes to be committed.

Cuidado com o **git add .**: ele leva tudo, inclusive arquivos que você não queria versionar. Veremos como evitar isso com o **.gitignore**.

git commit: tira a foto e cola no diário

Git Bash

```
$ git commit -m "Cria a pagina inicial"
[main (root-commit) 3f9a1c2] Cria a pagina inicial
 1 file changed, 12 insertions(+)
 create mode 100644 index.html
```

Grava no Cofre tudo o que está na Doca. O **-m** é a mensagem: uma frase curta, no imperativo, dizendo **o que** a foto registra. O código entre colchetes (**3f9a1c2**) é o início do hash do commit.

Octocat

Dica de quem já viu muito commit: escreva a mensagem como se completasse a frase “Este commit...”. “Cria a página inicial” funciona. “Mudanças” e “aaa” não ajudam ninguém.



git log: a linha do tempo

Git Bash

```
$ git log --oneline
a07f6b1 (HEAD -> main) Registra o dia 2
5d2c9e4 Registra o dia 1
3f9a1c2 Cria a pagina inicial
```

Variação	O que mostra
git log	Cada commit completo: hash inteiro, autor, data e mensagem
git log --oneline	Uma linha por commit, com o hash curto. É a mais usada
git log --oneline --graph --all	O desenho das branches de todas as linhas. Ótimo a partir do cap. 7
git log -p	Cada commit junto com as linhas que ele alterou

git diff: o que mudou?

Git Bash

```
$ git log --oneline
diff --git a/index.html b/index.html
@@ -3,5 +3,6 @@
   <head>...</head>
   <body>
     <h1>Diario de Bordo</h1>
+    <p>Dia: decolagem.</p>
   </body>
   </html>
```

- **git diff** compara o Hangar com a Doca: mostra o que mudou e **ainda não foi preparado**.
- **git diff --staged** compara a Doca com o último commit: mostra o que vai entrar no próximo commit.
- **git diff <hash1> <hash2>** compara dois commits quaisquer.

Linhas com + foram adicionadas, e com -, removidas.

Lendo o git status

O Git diz	Significado
On branch main	Você está na branch main
No commits yet	Ainda não existe nenhum commit
Untracked files	Arquivos novos que o Git ainda não conhece (estão só no Hangar)
Changes not staged for commit	Arquivos conhecidos que mudaram, mas não foram para a Doca
Changes to be committed	Arquivos na Doca, prontos para o próximo commit
nothing to commit, working tree clean	Tudo salvo. Não há nada novo

Circuito Interno

.gitignore

Nem todo arquivo deve ser versionado (senhas, pastas de dependências, arquivos temporários). Crie um arquivo chamado **.gitignore** na raiz do projeto e escreva, um por linha, o que o Git deve ignorar:

.gitignore

```
.env
node_modules/
*.log
```

O assunto será aprofundado na Apostila Completa

SOS do primeiro voo

Quando algo trava...

O que apareceu	O que fazer
Please tell me who you are / Author identity unknown	O Git não sabe o seu nome. Rode os dois **git config --global** do capítulo 2 e refaça o commit.
Abriu uma tela estranha (o Vim)	Você esqueceu o **-m**. Aperte **Esc**, digite **:q!** e Enter para sair sem commitar. Depois refaça com **-m**.
fatal: not a git repository	Você não está na pasta do projeto, ou esqueceu o **git init**. Rode **pwd** e **ls -a** para ver onde está.
nothing to commit, working tree clean	Não há mudança nova. Confira se salvou o arquivo no editor (Ctrl+S) antes do **git add**.

Teste Relâmpago

1. Qual comando mostra em que área cada arquivo está?

a) **git log**
b) **git status**
c) **git init**
d) **git diff <hash>**

2. Qual comando leva um arquivo do Hangar para a Doca?

a) **git commit**
b) **git add**
c) **git status**
d) **git log**

3. O que acontece se você rodar git commit sem -m?

a) Nada
b) O commit sai sem mensagem
c) Abre um editor de texto (Vim) para você escrever a mensagem
d) O Git apaga o arquivo

4. Para ver as linhas que mudaram e ainda não foram preparadas, use:

a) **git diff**
b) **git log -p**
c) **git init**
d) **git add .**

Cápsula do capítulo

- **git init** cria o repositório, uma vez por projeto.
- **git status** é a bússola: use sempre.
- **git add** leva à Doca e **git commit -m** grava no Cofre
- **git log --oneline** mostra a linha do tempo e **git diff** mostra o que mudou.
- O Git não cria arquivos: ele acompanha o que você cria.

Missão 05

A Estação Espacial GitHub

Levar o seu projeto para a nuvem: remote, push, clone e pull.

Objetivo	Publicar o seu projeto no GitHub e sincronizar
Comandos novos	remote, push, pull, clone, fetch



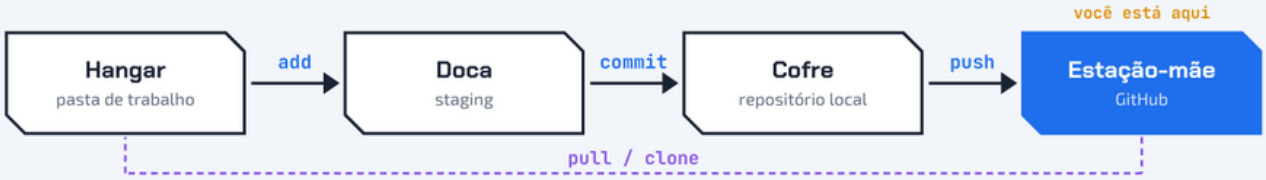


Octocat

Até agora tudo morava no seu computador. Hoje o projeto ganha endereço na nuvem: backup, vitrine e ponto de encontro com outras pessoas, tudo de uma vez.

Local x Remoto

O seu repositório no computador é o **local**. A cópia na nuvem, no GitHub, é o **remoto**. Um **remote** é só o endereço (a URL) de um repositório remoto, guardado com um apelido. O apelido padrão é **origin**.



Tradutor Galáctico

Termo real	Metáfora	Onde a metáfora falha
Remote / origin	A frequência de rádio da Estação-mãe	"origin" não é um nome mágico: é só o apelido que o Git dá ao primeiro remoto. Você pode trocar ou ter vários.

Criando o repositório no GitHub

1. Entre no GitHub e clique no +, no canto superior direito, e em **New repository**.
2. Dê um nome sem espaços, como **diario-de-bordo**.
3. A descrição é opcional, mas ajuda quem chega ao projeto.
4. Escolha P**ublic** (qualquer pessoa vê) ou **Private** (só você e quem você convidar).
5. **Não** marque "Add a README" se o seu projeto já tem commits locais. Isso evita um conflito na primeira conexão.
6. Clique em **Create repository** e copie a URL HTTPS que aparece.

Cuidado

Repositório **público** é visível para o mundo inteiro. Nunca envie senhas, chaves de acesso ou arquivos pessoais. No capítulo 12 você vê como o **.gitignore** ajuda.

Conectando e enviando: o primeiro push

Git Bash

```
$ git remote add origin https://github.com/seu-usuario/diario-de-bordo.git
$ git branch -M main
$ git push -u origin main
Enumerating objects: 9, done.
Writing objects: 100% (9/9), done.
To https://github.com/seu-usuario/diario-de-bordo.git
  * [new branch] main -> main
branch 'main' set up to track 'origin/main'.
```

Comando	O que faz
git remote add origin <url>	Registra o endereço da nuvem com o apelido origin
git branch -M main	Garante que a sua branch se chama main
git push -u origin main	Envia os commits. O -u liga a sua main à main remota, e depois basta git push
git remote -v	Lista os remotos configurados

No primeiro push, o Git para Windows abre o navegador para você entrar no GitHub. Atualize a página do repositório: os arquivos e a lista de commits estão lá.

Trazendo de volta: clone, fetch e pull

Comando	O que faz
git clone <url>	Baixa uma cópia completa de um repositório, com o histórico, e já configura o origin. É assim que você começa a mexer em um projeto que já existe
git fetch	Baixa as novidades do remoto, mas não mexe nos seus arquivos
git pull	Baixa as novidades e já junta com a sua branch (é um fetch seguido de merge)

Git Bash

```
$ git pull
Updating a07f6b1..d9e8f21
Fast-forward
  README.md | 3 +++
```

Octocat

Hábito de ouro: antes de começar a trabalhar, rode git pull. É o jeito mais barato de evitar conflitos e dor de cabeça.



README: o cartão de visitas

O arquivo README.md na raiz do projeto aparece na página do repositório. Ele explica o que o projeto é, como usar e como contribuir. Escreve-se em Markdown, um jeito simples de formatar texto:

Você escreve	Resultado
`# Título`	Título grande
`## Subtítulo`	Título de seção
`**negrito**`	**negrito**
`- item`	Item de lista
`` `código` ``	Trecho de código
`[texto](https://exemplo.com)`	Link

Seu perfil também é vitrine. O GitHub mostra os seus repositórios, os projetos fixados e o gráfico de contribuições, aquele mosaico verde. Se criar um repositório com o mesmo nome do seu usuário e um README, o texto aparece no topo do seu perfil.

Protocolo de desembarque (computador compartilhado)

1. Confira que o trabalho está no GitHub (`git status` sem pendências e `git push` feito).
2. Saia da sua conta no navegador (Sign out).
3. No Windows, abra o **Gerenciador de Credenciais**, vá em Credenciais do Windows e remova as entradas que começam com `git:https://github.com`.
4. Apague a pasta do projeto do computador.

SOS da estação

Quando o envio falha...

O que apareceu	O que fazer
`rejected` / `failed to push some refs`	O remoto tem commits que você não tem. Rode `git pull` e depois `git push` de novo.
`refusing to merge unrelated histories`	Você criou o repositório com README no GitHub e também fez commits locais. Rode `git pull origin main --allow-unrelated-histories` e resolva o que aparecer.
`remote origin already exists`	Já existe um origin. Veja com `git remote -v` e troque a URL com `git remote set-url origin <url>` .
`Authentication failed` / `Permission denied`	O login falhou. Tente de novo pelo navegador, confira se o repositório é seu ou use um token de acesso pessoal.

Teste Relâmpago

1. Qual comando baixa uma cópia completa de um repositório remoto?

a) **git push**
b) **git clone <url>**
c) **git init**
d) **git add .**

2. O que significa origin?

a) O primeiro commit
b) O apelido do repositório remoto principal
c) Uma branch
d) O nome do autor

3. O que o -u faz no git push -u origin main?

a) Apaga o remoto
b) Liga a sua main à main remota, para usar só git push depois
c) Desfaz o commit
d) Atualiza o Git

4. O push foi rejeitado. O primeiro passo é:

a) **git push --force**
b) **git pull**
c) Apagar o repositório
d) **git reset --hard**

Cápsula do capítulo

- Local é o seu computador; remoto é o GitHub. O origin é o apelido do remoto.
- **git remote add origin <url>** e **git push -u origin main** publicam o projeto.
- **git clone** baixa um projeto; **git pull** traz as novidades. Faça pull antes de trabalhar.
- README.md é o cartão de visitas do repositório.

Oficina de Git e GitHub

COMMIT ZERO

Nunca usou Git? Ótimo: esta apostila começa do zero. Com o Octocat como copiloto, você instala o Git, entende como ele guarda o seu trabalho, faz os seus primeiros commits e publica o projeto no GitHub. Feita para ser lida com o terminal aberto ao lado e guardada para consultar sempre que bater a dúvida.

O que você vai aprender

- Instalar, configurar e dar os primeiros commits
- Entender como o Git guarda o seu trabalho por dentro
- Usar o ciclo init, status, add, commit, log e diff
- Enviar o seu projeto ao GitHub com push, clone e pull
- Sair de apuros com as tabelas de SOS dos erros mais comuns

Matheus Santos Gomes

Edição Oficina 2026

Contato: msamtosgomes42@gmail.com

Material independente, sem vínculo com o GitHub. GitHub e Octocat são marcas registradas do GitHub, Inc. Artes do Octocat: Octodex (octodex.github.com), usadas sem alterações; crédito aos artistas na página de créditos. O Git é um software livre.